CONTEXT ENGINEERING IN RAG SYSTEMS

Maha Ibrahim , Reem Abdulaleem , Seif Hamdan , Abdullah Harhar

CONTENTS

LLM LIMITATIONS

LARGE LANGUAGE MODELS LIKE CHATGPT ARE POWERFUL, BUT THEY HAVE TWO CRITICAL FLAWS

Hallucination

LLMs confidently generate wrong answers when they don't know something. They don't say "I don't know", they make something up.

Knowledge Cutoff

LLMs are trained on data up to a certain date. Anything after that simply doesn't exist for them.

WHY DO LLMS HALLUCINATE?

LLMs don't actually know facts the way humans do. They are trained to predict the next most likely word based on patterns in training data.

This means when they don't have the answer they don't stop — they just keep predicting the most statistically likely response. That response sounds confident and fluent but can be completely wrong.

Three main reasons it happens:

1. The answer wasn't in the training data
2. The training data had conflicting information
3. The question is too specific

WHAT IS RAG?

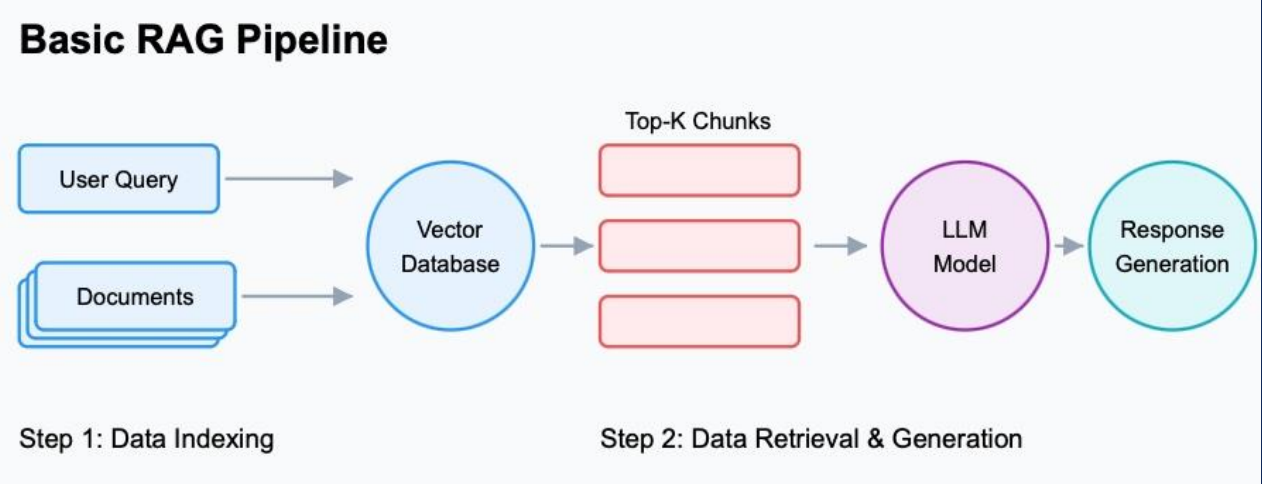
RETRIEVAL-AUGMENTED GENERATION
RAG GIVES THE LLM ACCESS TO REAL DOCUMENTS BEFORE IT ANSWERS.

Retrieval:

When a user asks a question, the system searches a database of documents and finds the most relevant chunks of text.

Generation:

Those chunks are inserted into the prompt, and the LLM generates an answer based on actual retrieved information — not guesswork.



WHY RAG ACTUALLY FIXES THIS

RAG fixes hallucination by changing what the LLM has to work with.

- Without RAG the LLM only has its training weights
- With RAG the LLM receives actual retrieved text in the prompt. Now instead of predicting from memory it is reading and summarizing real information.

Why this works:

- The LLM is actually very good at reading and summarizing text and that's what it was trained on
- You are replacing its weakest skill (remembering specific facts) with its strongest skill (understanding and summarizing text it can see)
- The answer is now grounded in a real source that can be checked

RAG VS NO RAG

Without RAG: Question: "What are the latest EU AI regulations?" Answer: *(LLM makes something up or gives outdated info)* Result: Hallucination, wrong date, missing details

With RAG: Question: "What are the latest EU AI regulations?" System retrieves actual EU AI Act documents Answer: Accurate, sourced, up to date

WHAT ARE EMBEDDINGS?

TURNING TEXT INTO NUMBERS

Computers can't understand text directly, they work with numbers.

Embeddings solve this.

An embedding is a list of numbers (a vector) that represents the meaning of a piece of text.

The key idea: **similar meaning = similar numbers**

Example:

- "Dog" and "Puppy" → vectors that are very close
- "Dog" and "Quantum Physics" → vectors that are far apart

This is how the system finds relevant chunks , not by matching exact words, but by matching meaning.

How embeddings are actually created:

- Text is tokenized first — split into smaller units called tokens
- Tokens are passed through a neural network
- The network outputs a fixed size vector
- The vector captures semantic meaning based on context not just the word itself

Embedding techniques:

- Word2Vec — older method, creates one vector per word, doesn't consider context
- GloVe — similar to Word2Vec, based on word co-occurrence statistics
- BERT based embeddings — considers full sentence context, same word gets different vector in different sentences
- Sentence Transformers — optimized specifically for sentence and paragraph level embeddings, what we use in our project

Why sentence transformers:

- all-MiniLM-L6-v2 is our model
- It produces 384 dimensional vectors
- Trained specifically to make similar sentences have similar vectors
- Fast and lightweight compared to larger models

VECTOR DATABASES

STORING AND SEARCHING EMBEDDINGS

Once we convert our documents into embeddings, we need somewhere to store them. That's what a vector database does.

FAISS (by Meta) and ChromaDB are two popular options.

When a user asks a question:

- The question is also converted to an embedding
- The vector DB finds the stored chunks whose embeddings are closest to the question
- It returns the top-k most relevant chunks (e.g. top 3 or top 5)

  These chunks are then handed to the LLM.

VECTOR DATABASES

- Document loading — read raw files (PDF, HTML, text)
- Text extraction — pull plain text from the format
- Chunking — split into smaller pieces with overlap
- Embedding generation — convert each chunk to a vector using embedding model
- Storage — save vector + original text + metadata in the database
- Indexing — build HNSW graph on top for fast search

What gets stored per chunk:

- The vector (384 numbers)
- The original text
- Metadata like source, chunk ID, page number

INDEXING

INDEXING ORGANIZES THE VECTORS IN A SMART STRUCTURE SO SIMILARITY SEARCH CAN BE DONE IN MILLISECONDS INSTEAD OF SECONDS.

- Flat Index — compares the query to every vector. Most accurate but slowest. Only practical for small datasets.
- IVF (Inverted File Index) — groups vectors into clusters. Only searches the most relevant clusters instead of everything. Much faster.
- HNSW (Hierarchical Navigable Small World) — builds a graph structure between vectors. Very fast and very accurate. Most popular in production systems.
- PQ (Product Quantization) — compresses vectors to save memory. Trades a little accuracy for much faster search and lower storage cost.

FAISS and ChromaDB both support multiple index types. Choosing the right one depends on how much data you have and how fast you need results.

INDEXING

How HNSW works:

- Builds multiple layers of graphs
- Top layers have few nodes with long range connections
- Bottom layers have all nodes with short range connections
- Search starts at the top layer and navigates down
- Like a highway system — start on the motorway, exit to local roads to find exact location

Why this matters for our project:

- Without indexing: searching 100 chunks means 100 comparisons
- With HNSW: search takes logarithmic time, scales to millions of chunks

SEMANTIC SEARCH

HOW THE SYSTEM FINDS RELEVANT TEXT

To measure how similar two vectors are, we use similarity metrics.

1) Cosine Similarity — measures the angle between two vectors. The smaller the angle, the more similar the meaning. This is the most commonly used method.
2) Manhattan Distance — measures distance by summing absolute differences
3) Euclidean Distance — straight line distance between two points in vector space
4) Dot Product — measures similarity by multiplying the values of two vectors together and summing them. Fast and simple. Works best when vectors are normalized. Often used in combination with cosine similarity.

For RAG systems, cosine similarity works best because it focuses on direction (meaning) rather than magnitude (length of text).

SEMANTIC SEARCH

Cosine Similarity : $\cos(\theta) = (A \cdot B) / (||A|| \times ||B||)$

- Where A and B are vectors, the dot product divided by the product of their magnitudes. Result between 0 and 1.

Manhattan Distance : $d = \Sigma|Ai - Bi|$

- Sum of absolute differences.

Euclidean Distance $d = \sqrt{\Sigma(Ai - Bi)^2}$

- Square root of sum of squared differences.

Dot Product : $A \cdot B = \Sigma(Ai \times Bi)$

- Multiply each dimension and sum them all up.

Example:

- Vector A = [1, 0]
- Vector B = [0.9, 0.1]
- Cosine similarity = very high, nearly 1
- These vectors point in almost the same direction = similar meaning

PROMPT ENGINEERING

PROMPT ENGINEERING TECHNIQUES

Prompt engineering is about how you phrase instructions to get the best output from an LLM.

1) **Zero-shot:** Just ask the question, no examples given. "*Summarize this text.*"
2) **Few-shot:** Give the model 2–3 examples before your real question so it understands the pattern. "*Here are two examples of summaries. Now summarize this.*"
3) **Chain-of-thought:** Tell the model to think step by step before answering. "*Let's think through this step by step...*" This dramatically reduces errors on complex questions.
4) **Tree-of-thought:** The model explores multiple reasoning paths and picks the best one. Good for complex problem solving.

Zero-shot:

- `Summarize the following text:`
- `The transformer architecture uses attention mechanisms...`

Few-shot:

- `Text: The sky is blue. Summary: Sky color description.`
- `Text: Water boils at 100 degrees. Summary: Water boiling point.`
- `Text: The transformer uses attention. Summary:`

Chain-of-thought:

- `Question: Is this student eligible for a scholarship?`
- `Let's think step by step:`
- `Step 1: Check if GPA meets the requirement...`
- `Step 2: Check enrollment status...`
- `Step 3: Check nationality requirement...`
- `Final answer:`

Tree-of-thought:

- `Question: Should we use FAISS or ChromaDB?`
- `Path A: If we prioritize speed → FAISS is better because...`
- `Path B: If we prioritize simplicity → ChromaDB is better because...`

WHAT IS CONTEXT ENGINEERING?

Context engineering is the practice of deliberately designing what information you give the LLM and how you structure it.

In RAG, after you retrieve relevant chunks, you still have to decide:

- How many chunks to include?
- In what order?
- Should you include the original question again?
- Should you add instructions like "only answer based on the provided context"?
- How do you format the prompt template?

Bad context engineering means the LLM ignores your retrieved docs, gets confused, or gives vague answers — even when you retrieved the right information.

Good context engineering means the LLM uses exactly what you gave it and produces a precise, grounded answer.

PROMPT EXAMPLES

Example 1 — Zero-shot fail (no retrieval):

- Question: "What is SRH Berlin's current tuition fee?"
- Without RAG: LLM guesses or gives outdated info
- Result: Hallucination

Example 2 — With RAG (success):

- Same question, but system retrieves the current SRH fee page Prompt includes retrieved text
- Result: Correct, specific answer

Example 3 — Chain-of-thought with RAG:

- Question: "Based on the retrieved policy, is a student eligible for a fee reduction?"
- Prompt: "Let's think step by step using only the provided context..."
- Result: LLM reasons through the policy correctly instead of jumping to a conclusion

SUMMARY

Today we covered:

- Why LLMs need RAG
- How RAG works
- How embeddings and vector databases power the search
- How prompt engineering and context engineering shape the final answer

Next steps for our project:

- Build the actual RAG pipeline in Python
- Connect a vector database (FAISS or ChromaDB)
- Build a simple chat interface
- Show a real comparison: question that fails without RAG, succeeds with RAG

THANK YOU